

# VOLE-ACT: Post-Quantum Anonymous Credit Tokens from VOLE-in-the-Head and MAYO

Samuel Schlesinger

July 2026

Research prototype documentation — not yet proven or independently audited

## Abstract

Anonymous credit tokens let an issuer grant a client a hidden, spendable balance: the client can later redeem credits unlinkably, each redemption consumes a one-time nullifier and returns a fresh token on the remaining balance. Classical constructions rest on discrete-logarithm-style algebra — blind or randomizable signatures, algebraic MACs, Sigma protocols — none of which survives a quantum adversary, and the post-quantum blind-signature literature has proved fragile. We describe VOLE-ACT, a post-quantum instantiation of the Katz–Schlesinger anonymous-credit design whose entire symmetric layer is Keccak and whose only algebraic assumption is the multivariate-quadratic (whipped MAYO) trapdoor. The central idea is to *avoid blindness entirely*: the issuer signs a revealed SHAKE256 commitment digest whose preimage stays hidden, and possession of the signature is later proven in zero knowledge inside a VOLE-in-the-Head proof system that is native to exactly the two ingredients we must prove — Keccak (a Boolean circuit) and MAYO verification (a quadratic system over  $\mathbb{F}_{16} \subseteq \mathbb{F}_{2^{128}}$ ). We explain the credential design, the proof circuit — including a degree-16 Keccak checkpointing technique that commits the permutation state only every four rounds — the quadratic-system constraint used for MQ verification, the exact 64-bit balance arithmetic, and the deferred-return settlement extension. We report implementation performance: on one Apple M4 Pro core-complex a spend proof takes  $\approx 19$ – $20$  ms to produce and  $\approx 6.5$ – $8$  ms to verify-and-sign, with 53 KiB proofs in the compact profile; tokens themselves are 283 bytes.

## 1 Introduction

An anonymous credit token system has one issuer and many clients. The issuer grants a client a public number of credits; the client can later spend part of that balance without revealing which issuance — or which earlier spend — produced the token being presented. Each accepted spend consumes a one-time nullifier (preventing double spending) and returns a fresh token carrying the remaining balance. The functionality generalizes anonymous tokens in the Privacy Pass style [?] from unit tokens to stateful balances, and specializes keyed-verification anonymous credentials [?] to a single numeric attribute with exact arithmetic.

Classically this is well-trodden ground: blind signatures [?], CL signatures [?], algebraic MACs [?] and their pairing-based relatives give compact, efficient constructions. All of them lean on the algebraic structure of discrete-log or pairing groups, both for the signature and for the efficient zero-knowledge presentation protocol. Neither survives a quantum adversary. The natural repair — a post-quantum *blind* signature — remains an active and unsettled research area

in which several early lattice-based proposals required substantial revision after cryptanalysis; we prefer a design that does not need blindness at all.

**The core idea: sign a digest, hide the preimage.** VOLE-ACT, following the Katz–Schlesinger design [?], sidesteps blindness. A credential is a short XOF stream

$$X = \text{SHAKE256}(\text{domain} \parallel \text{ctx} \parallel \kappa \parallel \text{enc}_{64}(b) \parallel \rho),$$

over a client-chosen 256-bit nullifier key  $\kappa$ , public balance  $b$ , and 256-bit hiding nonce  $\rho$ . The first  $4m$  bits of  $X$  form the *commitment digest*  $C \in \mathbb{F}_{16}^m$ ; the next 256 bits form the *nullifier*  $N$ . At issuance the client *reveals*  $C$  and proves in zero knowledge that it is well-formed for the public balance  $b$ ; the issuer then computes an ordinary MAYO preimage  $\sigma \leftarrow \text{SPre}(\text{sk}, C)$  and returns it. Nothing is blind: the issuer sees exactly the digest it signs. Unlinkability comes later — at spend time the client proves *in zero knowledge* that it knows  $(\sigma, \kappa, b, \rho)$  with  $P^*(\sigma) = C(\kappa, b, \rho)$ , revealing only the nullifier  $N$ , the public spend amount, and a fresh commitment digest  $C'$  for the successor token. The signature and the signed digest are never shown again, so there is nothing to link. The unrevealed suffix of  $X$  is treated as pseudorandom given its prefix, which is what makes the nullifier unlinkable to the issuance in which the issuer saw  $C$ .

This pattern reduces the required primitives to (i) a post-quantum signature whose *verification* is cheap inside a proof system, and (ii) a post-quantum zero-knowledge proof system that is efficient on *hash-heavy Boolean* statements. The pairing of MAYO with VOLE-in-the-Head is unusually harmonious on both counts:

- **MAYO** [?, ?] verification evaluates a public multivariate quadratic map over  $\mathbb{F}_{16}$ . Since  $\mathbb{F}_{16}$  embeds as a subfield of the proof system’s tag field  $\mathbb{F}_{2^{128}}$ , the whole verification equation becomes a single batched *quadratic system* constraint in the proof — no bit decomposition, no modular arithmetic emulation.
- **VOLE-in-the-Head** [?, ?] proof systems commit witness *bits* and prove Boolean and low-degree relations at QuickSilver [?, ?] cost: linear gates are free, and the entire batch of polynomial constraints of maximum degree  $d$  is checked with  $d$  published  $\mathbb{F}_{2^{128}}$  elements. Keccak — the only hash we use anywhere — is the friendliest possible circuit for such a system: its round function is degree 2.

The result is a scheme whose zero-knowledge layer is hash-based (analyzed in an ideal-XOF model), whose fiscal security rests on an adaptive *one-more-preimage* property of the whipped MAYO map (the issuer acts as a preimage oracle for adaptively chosen targets, so plain existential unforgeability is not the right game — see Section ?? and the closely related discussion in PoMFRIT [?]), and whose anonymity is designed to survive even a MAYO trapdoor break. There are no lattices and no pairings.

**Contributions of the implementation described here.** (1) A concrete credential and settlement design: one XOF stream yielding both commitment and nullifier; a typed two-format token state machine; and a *deferred-return* extension in which the issuer selects a bounded return  $t' \leq s$  only *after* verifying the spend proof, wrapped in a nested hash so the returned amount cannot be reopened or transplanted. (2) A proof circuit for SHAKE256 with *degree-16 checkpointing*: the permutation state is committed only every four rounds, quartering the dominant witness cost by exploiting the proof system’s polynomial constraint degree. (3) A

batched quadratic-system constraint that expresses the whipped MAYO verification equation with shared product terms across all  $m$  equations, folded by verifier randomness. (4) An engineering result: the full protocol runs at interactive latencies on commodity hardware (Table ??) with 283-byte tokens and 53 KiB compact spend proofs, using hardware Keccak, word-level bit-matrix transposition, and parallel GGM tree expansion.

## 2 Preliminaries

### 2.1 Notation

$\lambda = 128$  throughout.  $\mathbb{F}_2, \mathbb{F}_{16} = \mathbb{F}_2[x]/(x^4+x+1)$  and  $\mathbb{F}_{2^{128}} = \mathbb{F}_2[x]/(x^{128}+x^7+x^2+x+1)$  are the Boolean, MAYO, and tag fields;  $\iota : \mathbb{F}_{16} \hookrightarrow \mathbb{F}_{2^{128}}$  is the protocol-specified field embedding (the subfield is unique; the embedding is fixed deterministically, with the reduction polynomial family used by FAEST [?]). SHAKE256 is used for every PRG, commitment, and Fiat–Shamir hash, always with prefix-free domain separation and fixed-width or length-framed inputs [?].  $\text{enc}_{64}(\cdot)$  is 64-bit little-endian encoding.

### 2.2 VOLE-based zero knowledge and VOLE-in-the-Head

In VOLE-based ZK [?, ?], the prover holds, for each witness bit  $u_t \in \{0, 1\}$ , a tag  $V_t \in \mathbb{F}_{2^{128}}$ , and the verifier holds a global secret  $\Delta \in \mathbb{F}_{2^{128}}$  and per-coordinate *keys*

$$K_t = V_t + u_t \cdot \Delta,$$

a vector oblivious linear evaluation (VOLE) correlation. Wires are affine combinations of committed bits and are linearly homomorphic for free. A multiplication (degree-2) constraint  $a \cdot b = c$  is checked QuickSilver-style: the verifier forms  $B = K_a K_b + \Delta K_c$ , which the prover can match with a degree-1 polynomial in  $\Delta$  exactly when the constraint holds; a random linear combination with challenge weights  $\chi_i$  batches all constraints into one polynomial identity checked at  $\Delta$ . The generalization to degree- $d$  polynomial constraints costs  $d$  published coefficients and  $d - 1$  random VOLE masks in total for the whole batch.

VOLE-in-the-Head [?] makes this *publicly verifiable and non-interactive*: the prover generates the VOLE correlation itself from  $\tau$  GGM [?] seed trees of depth  $h$  (with  $\tau \cdot h = \lambda$ ), commits to all  $2^h$  leaves of each tree with an all-but-one vector commitment, and the Fiat–Shamir challenge  $\Delta$  selects, per tree, one leaf that stays hidden; the remaining leaves are opened and the verifier recomputes its keys. Soundness comes from the hidden leaf; zero knowledge from the fact that the opened leaves are exactly what the verifier can recompute anyway. Our instantiation follows the hardening of FAEST [?] and “One Tree to Rule Them All” [?]: every PRG and commitment call binds a per-proof  $2\lambda$ -bit salt, a per-tree derived salt, and the exact tree position, preventing the known multi-target and multi-instance amortization attacks on the 128-bit seeds; small-field consistency across the  $\tau$  trees uses public corrections re-based onto one shared  $u$ -vector, in the style of SoftSpokenOT [?].

Three parameter profiles trade proof size against tree-expansion work: *compact*  $(\tau, h) = (16, 8)$ , *balanced*  $(32, 4)$ , and *low-latency*  $(64, 2)$ . Each opened proof reveals  $\tau$  tree openings of  $h$  sibling seeds, so proofs shrink as  $h$  grows while the prover expands  $\tau \cdot 2^h$  leaves.

**The consistency check must be a matrix hash.** Because each tree contributes only  $k$  bits of  $\Delta$ , a cheating prover who mangles one tree’s correction data is caught by a per-tree check

only with probability  $1 - 2^{-h}$  — as low as  $3/4$  for  $h = 2$ . The implementation therefore applies a 128-row *column-wise linear universal hash* to the entire tag matrix (two independent polynomial evaluations combined with random weights, plus a one-time-pad block), so that any inconsistency survives into the check equation except with probability  $\approx 2^{-\lambda}$  plus small degree-dependent evaluation terms, regardless of the chunk structure (an exact bound is part of the outstanding analysis). A scalar hash in place of the matrix hash is a real soundness bug we caught during development; we flag it because the failure is silent and parameter-dependent.

## 2.3 MAYO

MAYO [?, ?] is an oil-and-vinegar [?] trapdoor map: a public quadratic map  $P : \mathbb{F}_{16}^n \rightarrow \mathbb{F}_{16}^m$  vanishing on a secret  $o$ -dimensional oil space, with  $o < m$  deliberately too small for direct Kipnis–Shamir-style attacks; the “whipping” construction evaluates  $k_w$  correlated copies to stretch the oil space, so the effective (whipped) public map is  $P^* : \mathbb{F}_{16}^{k_w n} \rightarrow \mathbb{F}_{16}^m$ , matching the notation of the MAYO literature. The trapdoor samples preimages  $\sigma$  with  $P^*(\sigma) = y$  for arbitrary targets  $y$ ; this *hash-then-invert* usage (rather than MAYO’s message-signing API) is exactly what the scheme needs. We use the round-2 parameter set MAYO<sub>2</sub>:  $(n, m, o, k_w) = (81, 64, 17, 4)$ , so  $\sigma \in \mathbb{F}_{16}^{324}$  (162 bytes) and targets are  $m = 64$  nibbles (32 bytes). We note the risk posture explicitly: multivariate trapdoors have a difficult history — Rainbow fell to a practical attack in 2022 [?] — and MAYO’s whipping is comparatively young. The design compartmentalizes this risk: a MAYO break forges credentials but does *not* deanonymize past transactions, whose privacy rests only on the hash-based layer.

For the proof circuit, the whipped map is expanded once per public key into  $m$  upper-triangular forms  $G_a \in \mathbb{F}_{16}^{k_w n \times k_w n}$  with  $P^*(\sigma)_a = \sigma^\top G_a \sigma$ ; the circuit then proves  $m$  quadratic equations over the *same*  $\binom{k_w n + 1}{2}$  product terms (Section ??).

## 3 The scheme

### 3.1 Credentials and nullifiers from one XOF stream

Fix a 256-bit context  $\text{ctx}$  binding the issuer key (by hash of the expanded public map), application domain, parameter set, VOLE profile, and protocol version. The client samples  $\kappa, \rho \leftarrow \{0, 1\}^{256}$  and, for base balance  $b \in \{0, \dots, 2^{64} - 1\}$ , derives the stream

$$X = \text{SHAKE256}(\text{“credential”} \parallel \text{ctx} \parallel \kappa \parallel \text{enc}_{64}(b) \parallel \rho), \quad C = X[0 : 4m], \quad N = X[4m : 4m + 256].$$

For MAYO<sub>2</sub>,  $C$  is  $4m = 256$  bits; its width as a collision domain matters for fiscal security (Section ??). Both values come from a *single* sponge evaluation — the message fits one Keccak rate block — so proving both costs one in-circuit permutation. The commitment prefix  $C$  is revealed to the issuer exactly once (at signing time); the nullifier suffix  $N$  is revealed exactly once (at spending time). Under the standard modeling of the sponge, the suffix is pseudorandom given the prefix, which is what divorces the spend from the issuance.

### 3.2 Two token formats and the settlement state machine

A *direct* token is  $(\sigma, \kappa, b, \rho)$  with  $P^*(\sigma) = C(\kappa, b, \rho)$  and effective balance  $b$ . A *deferred-return* token additionally carries an issuer-chosen return  $t$ , with

$$P^*(\sigma) = D(\text{ctx}, C(\kappa, b, \rho), t) = \text{SHAKE256}(\text{“deferred”} \parallel \text{ctx} \parallel \text{pack16}(C) \parallel \text{enc}_{64}(t))[0 : 4m],$$

and effective balance  $b + t$  (exact, non-wrapping). The nested hash gives a simple, domain-separated binding of both the base commitment and the return: changing  $t$  requires a fresh preimage or a target collision, and no algebraic combiner (such as  $C + \iota(t)$ ) has to be analyzed for malleability at all.

Spends are typed. An ordinary **spend** consumes either format and always outputs a direct token; **spend-with-deferred-return** consumes either format and outputs a deferred-return token. The client commits to a fresh  $(k', \rho', b')$  *before* the issuer chooses  $t' \leq s$ , which is precisely the case that cannot be expressed as a smaller net spend — if the return were known in advance, the client would simply spend less. All four transitions carry distinct domain-separation tags end-to-end, so mode confusion at the byte level cannot create an untyped path.

### 3.3 The spend relation

To spend public amount  $s$  from a direct token, the client reveals  $(s, N, C')$  and proves knowledge of  $(\sigma, \kappa, b, \rho, \kappa', b', \rho')$  such that

$$P^*(\sigma) = C(\kappa, b, \rho), \quad N = N(\kappa, b, \rho), \quad b' + s = b, \quad C' = C(\kappa', b', \rho').$$

For a deferred-input spend the first equation becomes  $P^*(\sigma) = D(\text{ctx}, C(\kappa, b, \rho), t)$  with hidden  $t$ , together with the exact fold  $b + t = e$  and  $b' + s = e$ . The balance equations are *exact 64-bit unsigned additions with zero final carry*, proven over committed carry bits — there is no modular wraparound to exploit, and overspending is exactly the statement “the final carry is 1,” which the circuit rejects. The issuer verifies, signs the fresh digest  $(\sigma' \leftarrow \text{SPre}(\text{sk}, C'))$ , or the nested target for a deferred settlement), and atomically stores the request digest and response under  $N$ ; exact retransmissions replay the stored response, so a network failure cannot burn a balance.

## 4 The proof circuit

The relation above is almost entirely SHAKE256: a direct-input spend proves two hidden sponge evaluations (old stream, fresh stream), a deferred-input spend proves three (the nested target costs one more). Everything else — the MAYO equation and the balance arithmetic — is quadratic. This section describes the three techniques that make the circuit small.

### 4.1 Degree-16 Keccak checkpointing

Keccak- $f[1600]$ ’s round function is degree 2 (the  $\chi$  step is the only nonlinearity) [?]. The naive circuit commits all 1600 state bits after every round: 24 rounds  $\times$  1600 witness bits, with degree-2 constraints between layers. But witness bits are the dominant cost in a VOLE-style proof (each consumes a VOLE coordinate, enlarging both the correction data and every leaf-PRG expansion), while raising the *degree* of the batched QuickSilver check is cheap: a maximum degree  $d$  costs  $d$  published field elements and  $d - 1$  mask VOLEs *in total*.

We therefore commit the state only every *four* rounds. Between checkpoints the circuit carries symbolic polynomial expressions: after rounds 1, 2, 3, 4 the expressions have degree 2, 4, 8, 16, and the checkpoint constraint asserts (per bit) that the degree-16 expression equals the next committed state — a degree-16 polynomial constraint, the largest the implementation accepts. This quarters the per-permutation witness from  $24 \times 1600$  to  $6 \times 1600 = 9600$  bits. One permutation dominates each hidden SHAKE evaluation, so a direct spend’s witness is

about 21,700 bits and a deferred spend’s about 31,500 — versus roughly 79,000 and 118,000 without checkpointing. The prover-side cost is the schoolbook product of coefficient vectors in the expression layer, which is pure  $\mathbb{F}_{2^{128}}$  multiplication throughput ( $\approx 1.1\text{M}$  multiplications per permutation) and benefits from carry-less multiply instructions.

## 4.2 The quadratic-system constraint for MAYO

Verifying  $P^*(\sigma) = y$  means checking  $m = 64$  quadratic equations

$$\sum_{r \leq c} G_a[r, c] \sigma_r \sigma_c = y_a, \quad a \in [m],$$

over the *same*  $\approx \binom{325}{2} \approx 52,650$  products  $\sigma_r \sigma_c$ . Materializing each product as a committed wire would add 52,650 witness bits (times four, as these are  $\mathbb{F}_{16}$  values); instead the backend supports a *deferred quadratic system*: the prover records the values and tags of every term factor, and at challenge time the  $m$  equations are folded into a single QuickSilver degree-2 check by random weights  $\varphi_a \in \mathbb{F}_{2^{128}}$  drawn from the transcript, with coefficients lifted through the embedding  $\iota$ . An unsatisfied equation survives the fold except with probability  $2^{-\lambda}$ . The committed signature nibbles enter as bit wires ( $4 \cdot k_w n = 1296$  bits), and the fold costs the prover  $O(m \cdot |\text{terms}|)$  table-lookup XORs — data-independent, since the tables are indexed by public key coefficients. The same mechanism folds the 64 quadratic carry equations of each balance addition.

## 4.3 Balance arithmetic in $\mathbb{F}_2$

Additions are proven bitwise with committed carry vectors:  $a_i + b_i + c_i = s_i$  linearly (over  $\mathbb{F}_2$ , addition is XOR — an early design error we caught was writing balance equations additively over the tag field, where they say nothing about integers), and the carry recurrence  $c_{i+1} = a_i b_i + c_i(a_i + b_i)$  quadratically via the folded system. Asserting the final carry to be zero makes every balance equation an exact statement about 64-bit unsigned integers, so conservation is enforced bit-exactly rather than modulo anything.

# 5 Security discussion

We summarize the intended arguments as *conditional sketches*; the composed system has no complete security reduction, the statements below are weaker than theorems, and the implementation carries an explicit prototype disclaimer. Where the repository’s own analysis (DESIGN.md, SECURITY.md) states sharper caveats, those govern.

**Soundness / fiscal safety (sketch, with known error terms).** Conditionally on the VOLEitH extraction argument, the matrix consistency check, and the quadratic-system fold behaving as intended, an accepted spend implies knowledge of a MAYO preimage for a well-formed credential digest, correct nullifier derivation, and exact balance arithmetic. The concrete error is *not*  $2^{-\lambda}$ : the degree-16 batched check alone contributes up to  $17/2^{128} \approx 2^{-123.9}$ , before vector-commitment, consistency-hash, Fiat–Shamir, and composition terms.

Two further caveats are first-order. First, the credential digest is a collision domain of only  $4m = 256$  bits: an adversary who finds *two openings*  $(\kappa, b, \rho) \neq (\kappa', b', \rho')$  with the same prefix  $C$  can present the *one* issuer-supplied preimage  $\sigma$  under both openings — whose suffix nullifiers

will generally differ — double-spending, and (with distinct balances) creating value. This is a generic collision search: about  $2^{128}$  classically, and query complexity  $\Theta(2^{256/3}) \approx 2^{85.3}$  for an ideal quantum collision adversary [?] (under BHT’s demanding memory model). The repository’s security analysis records this ceiling; deployments requiring a higher quantum collision margin need a larger  $m$  or a widened digest. Second, the issuer acts as a *preimage oracle* for adaptively chosen, hash-shaped targets, so unforgeability must be assumed as an adaptive chosen-target *one-more-preimage* property of the whipped MAYO map — a stronger and less-studied assumption than MAYO’s standard signature unforgeability; see PoMFRIT [?] for a direct treatment of this distinction. Honest double spends are prevented operationally: a repeated credential reveals the same deterministic nullifier and is refused by the nullifier store, whose atomic retry semantics also make honest retransmission safe.

**Anonymity (sketch).** A spend reveals  $(s, N, C', \text{proof})$ . The intended argument is: the VOLEitH argument is honest-verifier zero knowledge and is compiled by salted Fiat–Shamir ( $2\lambda$ -bit per-proof salt);  $C'$  hides its opening under the 256-bit nonce  $\rho'$ ; and  $N$  is the unrevealed suffix of an XOF stream whose prefix the issuer saw at issuance, which is pseudorandom in an ideal-XOF model. None of these steps is currently backed by a proof for this exact composition (in particular, FIPS 202 standardizes SHAKE but supplies no such theorem), so the claim is: *in the ideal-XOF/ROM model, with high-entropy  $\kappa, \rho$* , linking a spend to its issuance reduces to the hash layer. In that regime a MAYO break that recovers trapdoors or forges preimages does not deanonymize past transactions. We stress the systemic caveats: spend amounts are public; direct-input and deferred-input proofs differ substantially in size and partition the anonymity set, as do credential kind, settlement mode, request length, timing, network identity, and retry behavior. The cryptographic layer bounds what the *proof* reveals, not what the deployment reveals.

**Multi-instance hardening.** Every PRG and commitment call in the tree layer binds salt and position (after [?]), preventing the known precomputation and batch-amortization attacks on the 128-bit seeds; a concrete multi-user bound still carries  $Q$ - and  $N$ -dependent terms. The wide consistency hash removes the  $2^{-h}$  per-chunk cheating strategy. Domain tags separate every hash use; all encodings are canonical and rejected on any alternate serialization.

**Implementation posture.** Field arithmetic is branch-free with no secret-indexed table lookups (the  $\mathbb{F}_{16}$  kernels operate on packed lanes with mask-based selection;  $\mathbb{F}_{2^{128}}$  multiplication uses carry-less multiply instructions), and MAYO’s Gaussian elimination runs on a fixed public schedule with masked selection. Secret-bearing heap state (VOLE tags, expression coefficients, recorded constraints, tree seeds) is zeroized on drop, and constraint storage is pre-sized so growth reallocation cannot copy secrets into unwiped freed memory. These are best-effort source-level properties: they do not cover stack and register copies, compiler transformations, aborting panics (and some pre-wipe panic windows), the prover’s remaining secret-dependent source branches, or physical side channels; the repository’s security notes track these openly, and complete constant-time behavior is unresolved.

## 6 Parameters, sizes, and performance

All measurements: research prototype, Rust, one Apple M4 Pro (14 logical cores; the parallel tree layer uses the default work-stealing pool), MAYO<sub>2</sub>,  $\lambda = 128$ . Figures are single-operation latencies on an otherwise idle machine — not sustained concurrent throughput — and Criterion benchmarks agree with the harness to within a few percent.

Table 1: Wire sizes (bytes). Tokens are profile-independent; only proof-carrying requests grow with the low-latency profiles.

| Object                         | compact<br>$(\tau, k)=(16, 8)$ | balanced<br>(32, 4) | low-latency<br>(64, 2) |
|--------------------------------|--------------------------------|---------------------|------------------------|
| Token (either format)          | 283                            | 283                 | 283                    |
| Issue request                  | 29 750                         | 55 286              | 106 358                |
| Spend request (direct input)   | 52 990                         | 101 726             | 199 198                |
| Spend request (deferred input) | 72 574                         | 140 894             | 277 534                |
| Issue / spend response         | 171 / 179                      | 171 / 179           | 171 / 179              |
| Public key (expanded map)      | 106 319                        | 106 319             | 106 319                |

Table 2: Operation latency, median of 15 (ms). “Verify-and-sign” is the complete issuer side including the MAYO preimage computation.

| Operation                                   | compact | balanced | low-latency |
|---------------------------------------------|---------|----------|-------------|
| Issue: client prove                         | 8.8     | 7.8      | 7.7         |
| Issue: issuer verify-and-sign               | 4.0     | 2.9      | 2.9         |
| Spend (direct): client prove                | 20.2    | 18.7     | 18.9        |
| Spend (direct): issuer verify-and-sign      | 8.0     | 6.5      | 6.5         |
| Spend (deferred): client prove              | 28.4    | 26.6     | 26.9        |
| Spend (deferred): issuer verify-and-sign    | 10.3    | 8.3      | 8.3         |
| Deferred settlement: issuer verify-and-sign | 8.0     | 6.5      | 6.4         |
| Issue: end-to-end                           | 13.9    | 11.6     | 11.5        |
| Spend (direct): end-to-end                  | 29.5    | 26.4     | 26.5        |

Notably, after optimizing the tree layer (parallel GGM expansion, streaming pairwise-halving plane accumulation, hardware Keccak via the ARMv8.2-SHA3 extension, and word-level bit-matrix transposition of the VOLE plane data), the compact profile’s latency penalty over low-latency has collapsed to  $\approx 6\text{--}7\%$  on direct-spend proving (other operations differ somewhat more) while its proofs are  $3.8\times$  smaller — a strong argument for compact, although the implementation’s shipped default remains balanced. The client prover, running once per transaction, is comfortably interactive at every profile.

## 7 Related work

VOLE-based designated-verifier ZK begins with Wolverine [?] and Mac’n’Cheese [?]; Quick-Silver [?] gives the degree- $d$  batched check we generalize. VOLE-in-the-Head [?] removes the



designated verifier and underlies the FAEST signature candidate [?], whose engineering (GGM trees, salting, consistency checking) we follow, with the hardening of [?]. MAYO [?, ?] descends from unbalanced oil-and-vinegar [?]; [?] is the cautionary tale for the assumption class. The closest post-quantum relative is PoMFRIT [?], which likewise combines VOLE-in-the-Head proofs of SHA-3-family circuits with the MAYO trapdoor and analyzes the stronger one-more-preimage property such issuers require; the constructions differ in functionality (VOLE-ACT’s stateful balances, typed settlement, and deferred returns). Anonymous tokens and credentials span blind signatures [?], CL signatures [?], keyed-verification credentials from algebraic MACs [?], and deployed unit-token systems [?]; VOLE-ACT is a post-quantum stateful-balance member of this family, instantiating the Katz–Schlesinger design [?].

## 8 Open problems

(1) A complete security reduction for the composed system (VOLEitH extraction + quadratic-system fold + the credential/nullifier stream) in the QROM. (2) A structured, non-materialized QuickSilver check for the whipped MAYO equation, removing the  $O(m(k_w n)^2)$  expanded-form table and enabling the larger MAYO parameter sets. (3) Hashing the compact public-key encoding (rather than the expanded forms) into the context — a transcript-breaking cleanup best done before any format freeze. (4) Aggregating or compressing spend proofs; at 53 KiB the compact profile is deployable server-side but remains the dominant bandwidth cost.

## A Implementation notes: optimization under a transcript-equality invariant

This appendix records the engineering that produced the numbers in Section ??, and the methodology we used to keep an aggressive optimization campaign honest about correctness. (Regression evidence of the kind described below constrains behavior on sampled inputs; it is not a proof, and it says nothing by itself about security or privacy — those rest on the analysis of Section ??.) None of this is a contribution to the *design*; we include it because the methodology and two negative results may be useful to other implementors of VOLE-style systems.

### A.1 Methodology

The protocol is deterministic given its RNG stream, and no optimization we accepted alters how randomness is consumed. We therefore maintained a *golden-hash corpus*: SHA3-256 digests of all 27 protocol artifacts (keys, requests, responses, tokens, across all three VOLE profiles) under fixed seeds. Every candidate optimization was admitted only if (i) it is arguable as a pure re-expression of identical arithmetic — XOR reassociation, bit-matrix transposition, precomputation of public values, order-preserving parallelism — (ii) the full test suite (including malicious-prover soundness regressions and kept-in-tree reference oracles for each rewritten kernel) passes, and (iii) every golden hash is byte-identical, regression-checking the emitted transcripts on the sampled corpus. Changes were benchmarked individually, hypothesis-first; two were rejected by measurement (below). A second, internal adversarial review pass over the accumulated diff (distinct from the still-outstanding external audit) caught two regressions the corpus could not see — a zeroization gap via container reallocation, and a panic reachable only through a contract-violating circuit — both fixed and regression-tested.

## A.2 What actually dominated

Profiling overturned our priors. Keccak hashing and  $\mathbb{F}_{2^{128}}$  multiplication — the “cryptographic” costs — were minor; the dominant costs were *data reshaping* and *memory management*: (1) assembling per-coordinate  $\mathbb{F}_{2^{128}}$  elements from  $\tau \cdot k$  bit planes with a branchy per-bit loop (18% of proving, 37% of verification); (2) roughly 600,000 small heap allocations per proof in the polynomial-expression layer; (3) per-element volatile zeroization of large secret buffers — and, embarrassingly, of 3.4 MB of *public* quadratic-system coefficients per proof. The effective fixes, in decreasing order of impact:

- **Word-level bit transposes.** Plane assembly and the consistency-hash column gathers are  $128 \times 64 / 128 \times 128$  bit-matrix transposes; the standard  $O(w \log w)$  word transpose replaced per-bit loops, incidentally removing a secret-dependent branch.
- **Inline expression storage.** Degree- $\leq 16$  coefficient vectors live in fixed inline arrays; constraint storage is pre-sized from the counting pass so growth reallocation can never copy secrets into unwiped freed memory.
- **A shared arena for public coefficients.** Quadratic-system coefficient tables are public-key material; storing them once (reference-counted views instead of per-term vectors) removed both the allocation traffic and their spurious per-proof zeroization.
- **Parallel and cheaper tree expansion.** The  $\tau$  GGM trees commit and reconstruct in parallel with order-preserving collection; the  $u$ /plane sums use a streaming binary-counter accumulation (pairwise subtree sums,  $\approx 2$  vector XORs per leaf instead of  $1 + k/2$ ).
- **Hardware and SWAR kernels.** Keccak uses the ARMv8.2-SHA3 extension where available;  $\mathbb{F}_{16}$  matrix kernels run eight lanes per word with mask-based selection (no lookup tables, preserving the constant-time contract);  $\Delta$ -powers are precomputed; the Keccak  $\rho\pi$  permutation is fused into  $\chi$ ’s indexing.

Cumulatively (Apple M4 Pro, balanced profile): issuer verify-and-sign  $21.4 \rightarrow 6.5$  ms, direct-spend end-to-end  $66.0 \rightarrow 26.4$  ms, compact client proving  $59.4 \rightarrow 20.2$  ms — and the compact profile’s latency penalty over low-latency collapsed from  $1.4\times$  to  $\approx 7\%$ , making the smallest proofs the sensible default.

## A.3 Negative results

Two plausible optimizations failed measurably and are recorded so they are not re-attempted from intuition. (1) Replacing per-field volatile zeroization with a whole-struct volatile write was *slower*: LLVM lowers large volatile aggregate stores poorly. (2) One specific AES-based leaf-expansion PRG — AES-128-CTR keyed per leaf (RustCrypto `aes 0.8 / ctr 0.9`, release build with LTO, Apple M4 Pro hardware AES) producing 4.2 KB per 16-byte seed — measured  $2.4\times$  *slower* than SHAKE256 once the ARMv8.2-SHA3 extension is enabled, for our workload shape. The construction we tested also does not by itself provide the salt-and-position binding of [?], which any AES instantiation must re-engineer via key derivation, tweaks, or extra calls — FAEST v2’s AES-based leaf commitments show this is achievable, at design-and-analysis cost. Since the properly hardened variant starts from a  $2.4\times$  deficit on this hardware, we retained the hash-based PRG; the hardening itself is a requirement, not a trade-off.

## References

- [1] W. Beullens. *MAYO: Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*. SAC 2021.
- [2] W. Beullens. *Breaking Rainbow Takes a Weekend on a Laptop*. CRYPTO 2022.
- [3] W. Beullens, F. Campos, S. Celi, B. Hess, M. J. Kannwischer. *MAYO: Algorithm Specification (Round-2 document, February 5, 2025)*. NIST PQC Additional Digital Signatures. This work pins the Round-2 specification; MAYO advanced to Round 3 in May 2026. <https://pqmayo.org/resources/>.
- [4] C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, P. Scholl. *Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head*. CRYPTO 2023.
- [5] C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, P. Scholl. *FAEST: Algorithm Specification, Version 2.0 (Round-2 document)*. NIST PQC Additional Digital Signatures. This work follows the Round-2 (v2.0) design; FAEST advanced to Round 3 in May 2026. <https://faest.info>.
- [6] C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, P. Scholl. *One Tree to Rule Them All: Optimizing GGM Trees and OWFs for Post-Quantum Signatures*. ASIACRYPT 2024.
- [7] K. Yang, P. Sarkar, C. Weng, X. Wang. *QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field*. ACM CCS 2021.
- [8] C. Weng, K. Yang, J. Katz, X. Wang. *Wolverine: Fast, Scalable, and Communication-Efficient Zero-Knowledge Proofs for Boolean and Arithmetic Circuits*. IEEE S&P 2021.
- [9] C. Baum, A. J. Malozemoff, M. B. Rosen, P. Scholl. *Mac’n’Cheese: Zero-Knowledge Proofs for Boolean and Arithmetic Circuits with Nested Disjunctions*. CRYPTO 2021.
- [10] L. Roy. *SoftSpokenOT: Quieter OT Extension from Small-Field Silent VOLE in the Minicrypt Model*. CRYPTO 2022.
- [11] O. Goldreich, S. Goldwasser, S. Micali. *How to Construct Random Functions*. Journal of the ACM 33(4), 1986.
- [12] A. Kipnis, J. Patarin, L. Goubin. *Unbalanced Oil and Vinegar Signature Schemes*. EUROCRYPT 1999.
- [13] NIST. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS PUB 202, 2015.
- [14] D. Chaum. *Blind Signatures for Untraceable Payments*. CRYPTO 1982.
- [15] J. Camenisch, A. Lysyanskaya. *An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation*. EUROCRYPT 2001.
- [16] M. Chase, S. Meiklejohn, G. Zaverucha. *Algebraic MACs and Keyed-Verification Anonymous Credentials*. ACM CCS 2014.
- [17] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, F. Valsorda. *Privacy Pass: Bypassing Internet Challenges Anonymously*. Proceedings on Privacy Enhancing Technologies 2018(3), 164–180.
- [18] G. Brassard, P. Høyer, A. Tapp. *Quantum Cryptanalysis of Hash and Claw-Free Functions*. LATIN 1998. arXiv:quant-ph/9705002.
- [19] C. Baum et al. *PoMFRIT*. USENIX Security 2026. IACR ePrint 2026/109.
- [20] J. Katz, S. Schlesinger. *Anonymous Credit Tokens*.